WG 4 - Francesca

Intelligent Systems

Exercise KG1: Rule-based reasoning: Recall exercise 13 from the previous working group. Prove or disprove whether a knowledge base KB={**C -> -(A v -B), A**} entail **-C** or not, but now using Forward Checking (FC) rule based reasoning. Give the rules that you apply, and write down the closure at the end:

RULES RECAP

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \quad \text{commutativity of } \wedge$$
$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \text{commutativity of } \vee$$
$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$
$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$
$$\neg(\neg\alpha) \equiv \alpha \quad \text{double-negation elimination}$$
$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{contraposition}$$
$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{implication elimination}$$
$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{biconditional elimination}$$
$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{de Morgan}$$
$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{de Morgan}$$
$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$
$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$

DEFINITIONS

FORWARD CHECKING

Applying rules to known facts to infer new information.

CLOSURE

KB* = { Initial/known facts + Inferred facts }

KB={C -> -(A v -B), A}

INFERRED FACTS:

- ELIMINATE IMPLICATION

  -C V -(A v -B) added to KB*

- DE MORGAN'S RULE

  -C v (-A & B) added to KB*

- DISTRIBUTE OR OVER AND

  (-C v -A) & (-C v B) added to KB*

- AND- ELIMINATION

  (-C v -A), (-C v B) added to KB*

KB* = { C -> -(A v -B), A, -C v -(A v -B), -C v (-A & B), (-C v -A) & (-C v B), (-C v -A), (-C v B)}

IS -C IN KB*?

IF NOT -> NOT ENTAILED

IF YES -> ENTAILED

In this case, -C is not in KB*: NOT ENTAILED.

Exercise 2: Rule-based reasoning: Look at the following variant of the previous exercise. Prove or disprove whether a knowledge base KB={**C -> -(A v -B), C**} entail **-A** or not, using Forward Checking (FC) rule based reasoning. Give the rules that you apply, and write down the closure at the end:

MODUS PONENS: AN IMPORTANT RULE

If a conditional statement ("if P then Q") is known to be true, and its antecedent (P) is also known to be true, then its consequent (Q) can be inferred to be true.

IS IT APPLICABLE IN OUR CASE? YES!

- conditional statement: $C \rightarrow \neg(A \vee \neg B)$
- the antecedent C, is also given as a fact in the KB

Applying Modus Ponens:

Since both the conditional statement and its antecedent are true, we can infer that the consequent $\neg(A \vee \neg B)$, is true can be added to the closure KB*

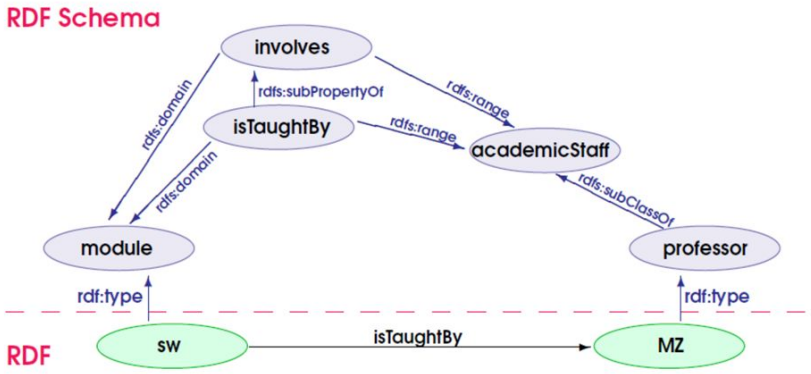
other Inferred Facts:

- MODUS PONENS
  $\neg(A \vee \neg B)$ added to KB*
- DE MORGAN'S RULE
  $\neg A$ & B added to KB*
- AND- ELIMINATION
  $\neg A$, B added to KB*

KB* = $\{C \rightarrow \neg(A \vee \neg B,\ C,\ \neg(A \vee \neg B), \neg A \ \&\ B,\ \neg A,\ B\}$

IS $\neg A$ IN KB*? YES, THEN IT IS ENTAILED!

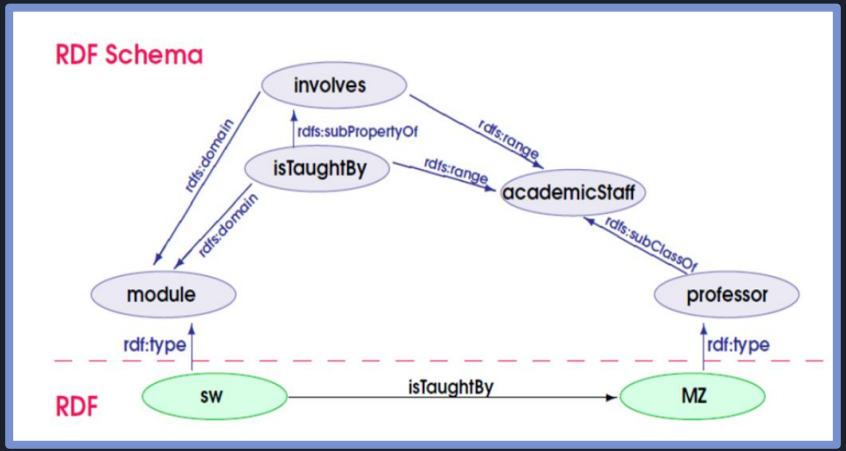
Exercise 3: Knowledge Graphs: Write down the following Knowledge Graph as a set of triples. Discuss whether there are any non-trivial triples that are entailed.



DEFINITION

TRIPLE

Structure composed of three elements, typically a subject, a predicate and an object



Set Of Triples:

- (sw, isTaughtBy, MZ)
- (MZ, rdf:type, professor)
- (professor, rdfs:subClassOf, academicStaff)
- (sw, rdf:type, module)
- (IsTaughtBy, rdfs:domain, module)
- (IsTaughtBy, rdfs:range, academicStaff)
- (IsTaughtBy, rdfs:subPropertyOf, involves)
- (involves, rdfs:domain, module)
- (involves, rdfs:range, academicStaff)

Rule to Infer Triples:

THE TYPE OF THE SUBCLASS OF A CLASS, IS THE TYPE OF THE CLASS AS WELL

Inferred Triples

- (MZ, rdf:type, academicStaff)

Exercise 3: Reasoning with RDFS rules: Suppose the following rule (only) is giving.

rdfs9: xxx rdfs:subClassOf yyy AND zzz rdf:type xxx ➔ zzz rdf:type yyy .

and an RDFS graph:

```
{ (ex:A rdfs:subClassOf ex:D),
 (ex:D rdfs:subClassOf ex:C),
 (ex:B rdfs:subClassOf ex:D),
 (ex:i1 rdf:type ex:A),
 (ex:i2 rdf:type ex:B)}
```

Which triples can be derived?

rdfs9: xxx rdfs:subClassOf yyy AND zzz rdf:type xxx → zzz rdf:type yyy .

{ (ex:A rdfs:subClassOf ex:D),
(ex:D rdfs:subClassOf ex:C),
(ex:B rdfs:subClassOf ex:D),
(ex:i1 rdf:type ex:A),
(ex:i2 rdf:type ex:B)}

- ex:i1 rdf:type ex:D

  ex:A rdfs:subClassOf ex:D & ex:1 rdf:type ex:A → ex:1 rdf:type ex:D

- ex:i2 rdf:type ex:C

  Apply rdfs9 twice!

  1. ex:B rdfs:subClassOf ex:D & ex:2 rdf:type ex:B → ex:2 rdf:type ex:D

  2. ex:D rdfs:subClassOf ex:C & ex:2 rdf:type ex:D → ex:2 rdf:type ex:C

WE CANNOT DERIVE (ex:B rdfs:subClassOf ex:C) WITH THIS RULE, EVEN THOUGH IT WOULD BE LOGICALLY CORRECT.

EXAM PREPARATION

Exercise 3: Graphs

Draw the graph ({a,b},{(a,a),(b,b)}). How many unconnected subgraphs are there? Write them down in a mathematically correct form. Is this a directed, or undirected graph?

Graph Notation and Structure:

- The graph is represented as a pair (V,E), where V is the set of vertices (or nodes) and E is the set of edges. In this case, $V=\{a,b\}$ represents the two nodes in the graph.
- The set of edges is $E=\{(a,a),(b,b)\}$. Each edge is represented as a pair, indicating a connection from the first element in the pair to the second.

UNCONNECTED SUBGRAPHS

DEFINITION

SUBGRAPH

Subset of a graph's vertices and edges.

UNCONNECTED SUBGRAPHS

Two subgraphs are unconnected if there are no edges between any of the vertices in one subgraph and any of the vertices in the other subgraph.

SOLUTION

there are two unconnected subgraphs because there are no edges connecting node *a* to node *b* or vice versa:

$$(a,\{(a,a)\}) \text{ and } (b,\{(b,b)\})$$

DIRECTED vs UNDIRECTED GRAPHS

DEFINITION

DIRECTED GRAPH

A directed graph has edges where the order matters; that is, an edge from node A to node B is different from an edge from node B to node A. In notation, these are different as $(A,B) \neq (B,A)$.

UNDIRECTED GRAPH

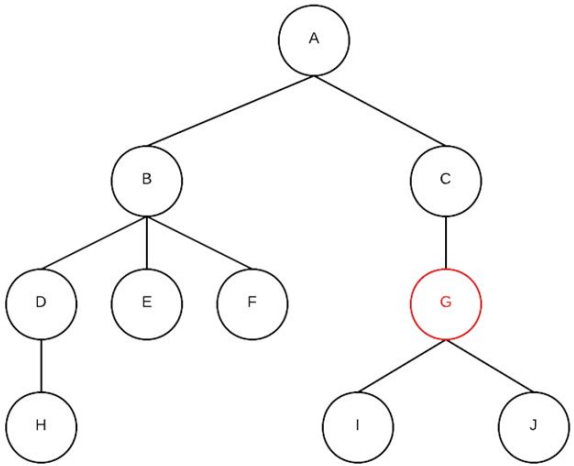
An undirected graph has edges where the order does not matter; that is, an edge between node A and node B is the same as an edge between node B and node A. They are typically represented as sets: $\{A,B\}=\{B,A\}$.

SOLUTION

Edges are represented as ordered pairs (in this case, an edge is directed from a vertex to the vertex itself), so it is a directed graph.

Exercise 5:

Let the following search tree, with an initial state A and a state G that satistifes the goal conditions.




```
function TREE-SEARCH(problem,fringe) return a solution or failure
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if EMPTY?(fringe) then return failure
        node ← REMOVE-FIRST(fringe)
        if GOAL-TEST[problem] applied to STATE[node] succeeds
            then return SOLUTION(node)
        fringe ← INSERT-ALL(EXPAND(node, problem), fringe)
```

Remember the algorithm for tree-search introduced in the lecture. Please follow the algorithm precisely (as if you were a Computer), given the following instructions. All the notes are added in inverse-alphabetic order in one group to the fringe (i.e. if node A is explored you add (C, B) to the fringe, either at the beginning if it is a FIFO queue, or at the end, if it is a LIFO list.

How many nodes do you need to explore before you find the goal state with DFS, BFD and IDFS? We count a node as explored when it is taken out of the fringe by the REMOVE-FIRST function.

DFS

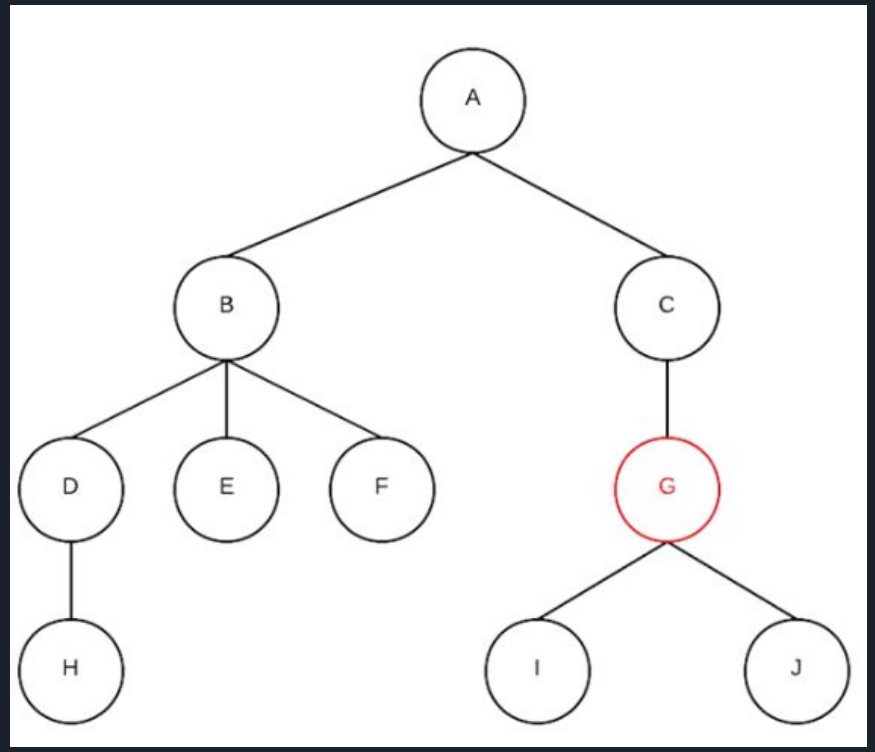
{A}

{C,B}

{G,B}

Goal reached!

3 NODES EXPANDED BEFORE THE GOAL!



BFS

{A}

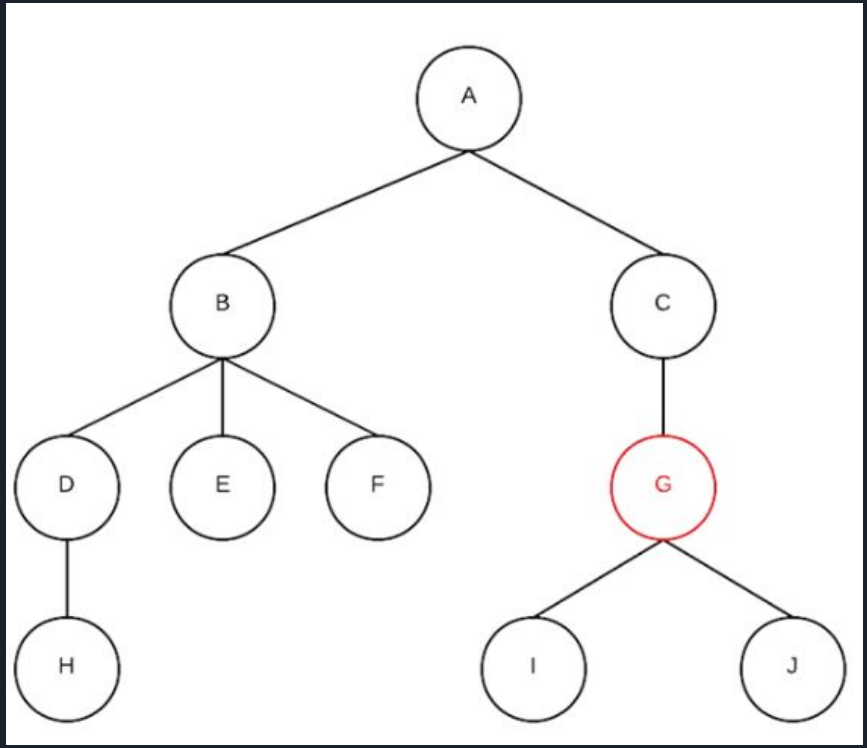
{C,B}

{B,G}

{G, F, E, D}

Goal reached!

4 NODES EXPANDED BEFORE THE GOAL!



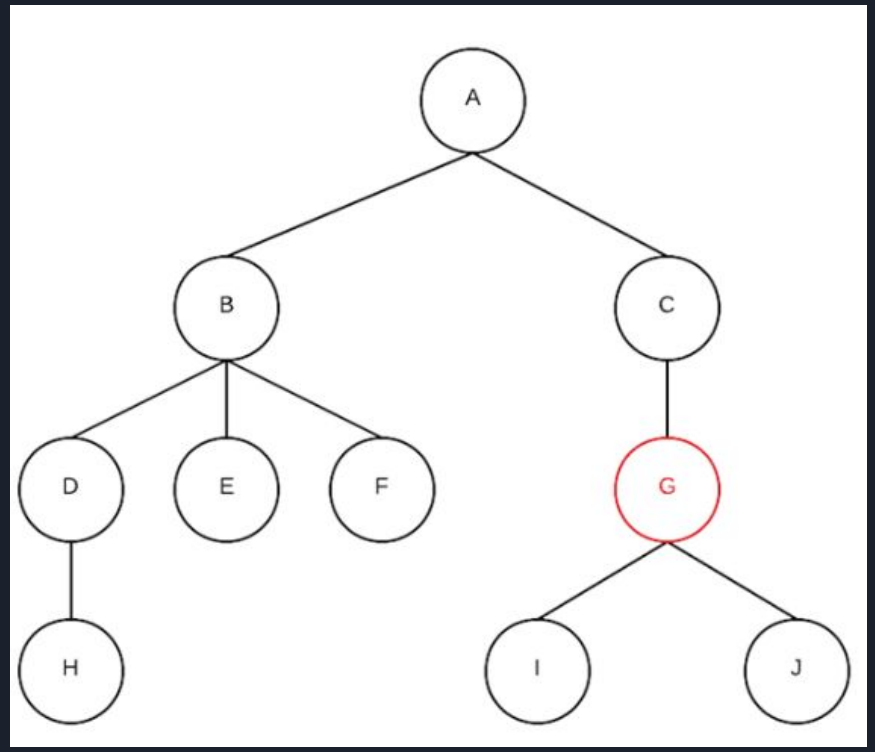
DFID

{A} 1 nodes +

{A,C,B} 3 nodes +

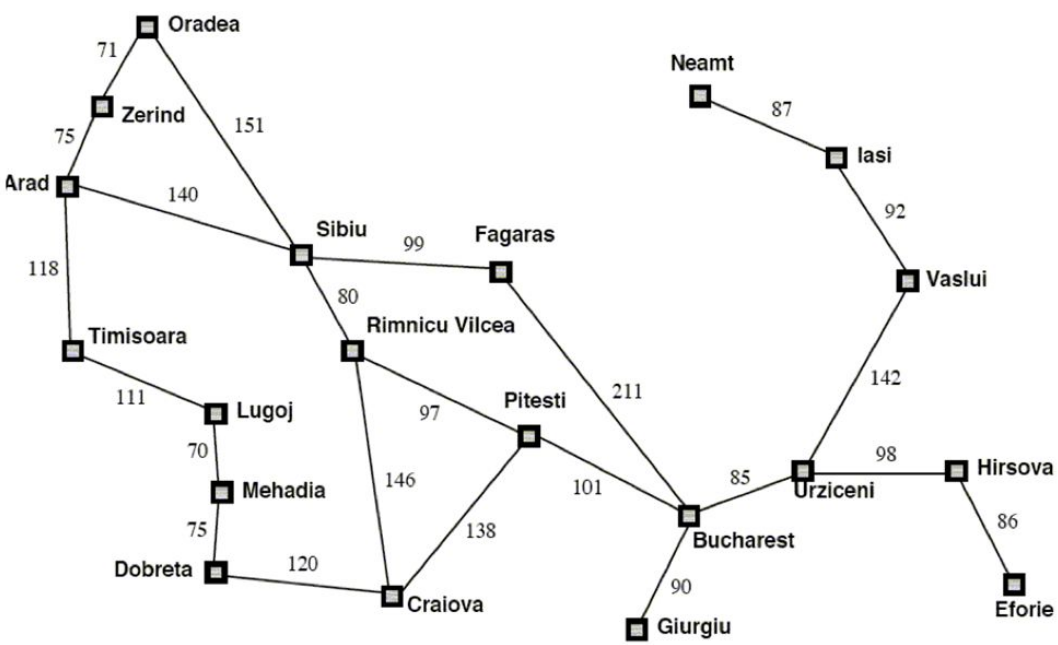
{A,C,G} 3 nodes

Goal reached!



7 NODES EXPANDED BEFORE THE GOAL!

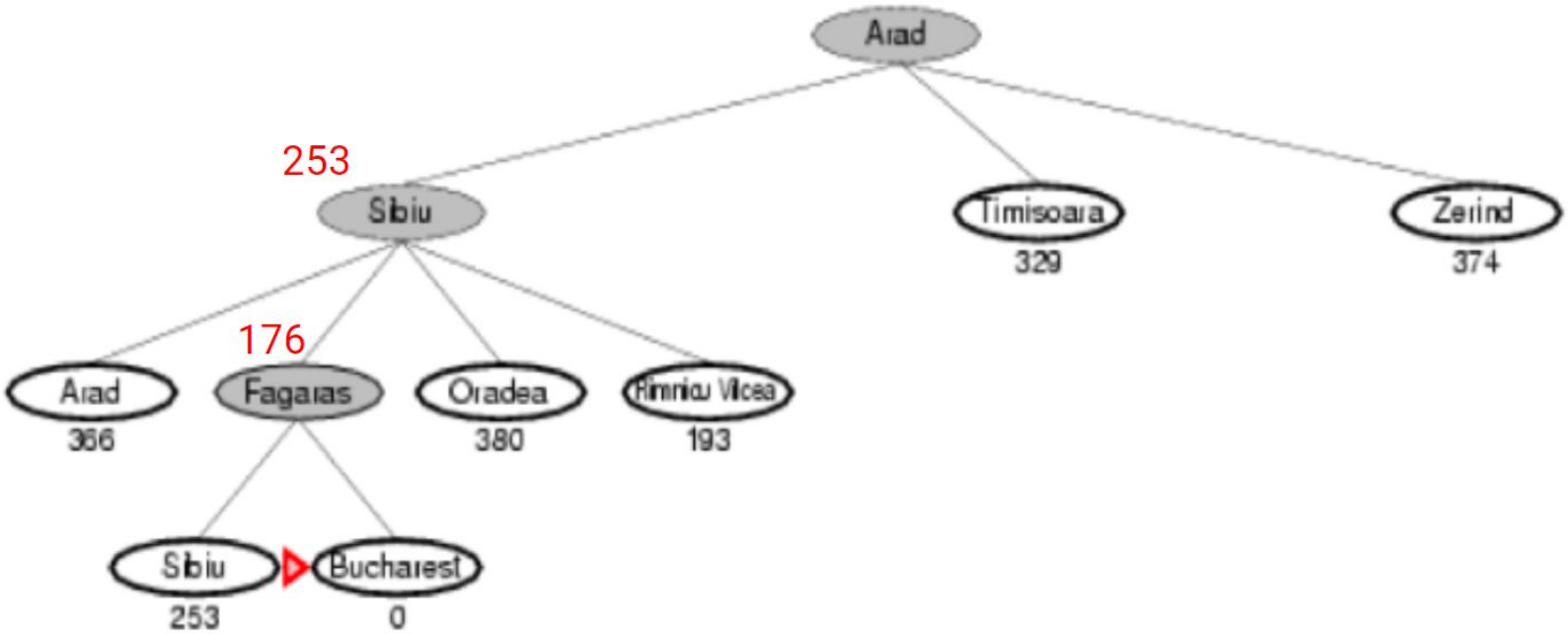
Hieronder vindt je een schematische kaart van Roemenië.



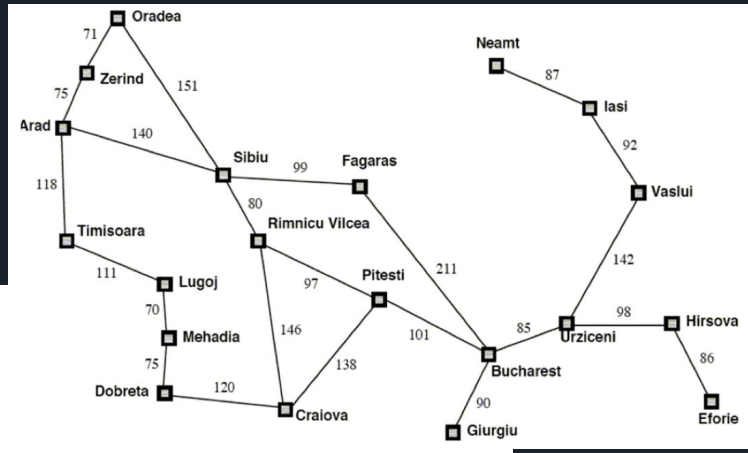
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnieu Vikea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

a. Show how a **hill-climbing** algorithm would search a route from Arad to Bucharest, where the straight-line-distance is used to estimate the town nearest to Bucharest. Support your answer with a search tree.
b. Do exactly the same as in question a), but now for a A/A* algorithm.
c. Is the applied algorithm in question b) an A or A* algorithm? Explain why.

HILL CLIMBING

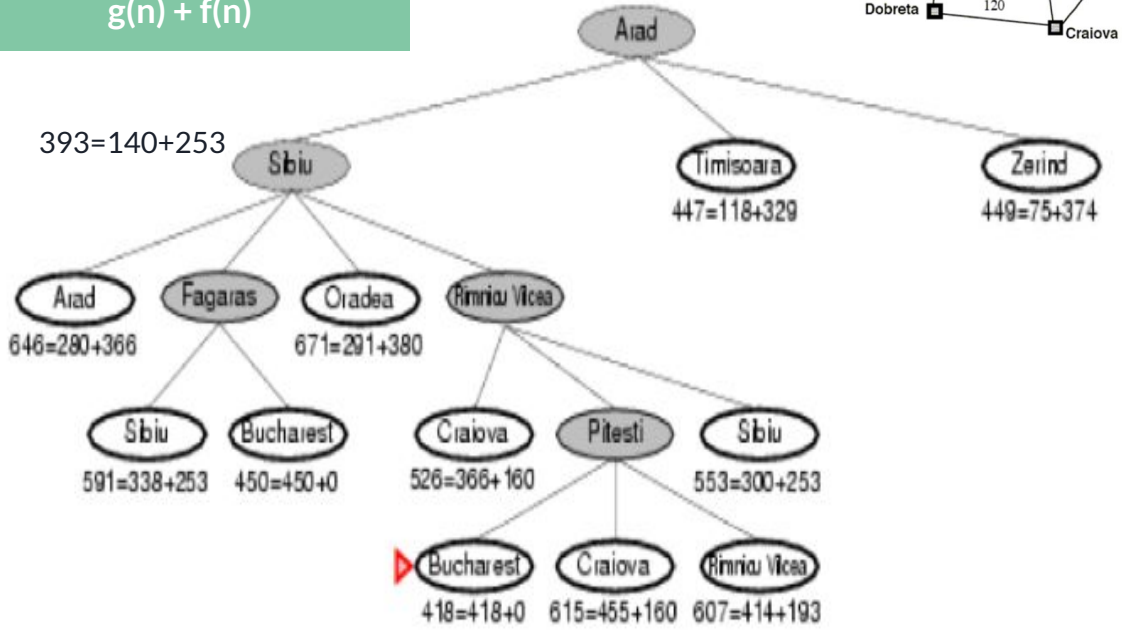


A/A* ALGORITHM



Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnieu Vikea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

EVALUATION FUNCTION:
g(n) + f(n)

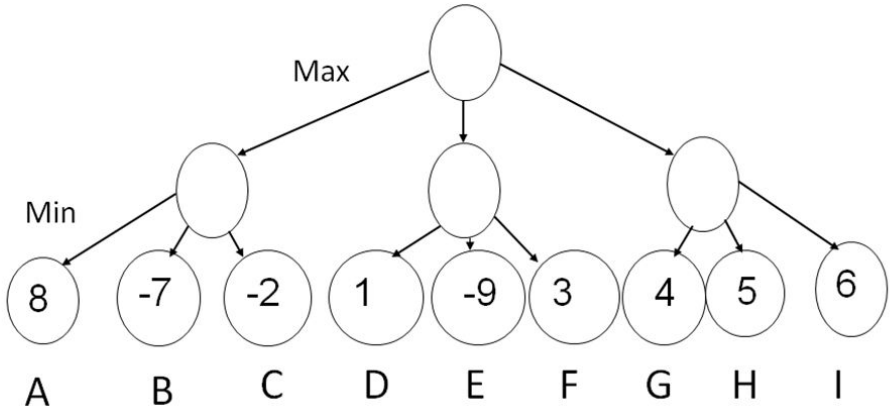


Is the applied algorithm in b) an A or A* algorithm?

A*, because estimated distance (straight-line distance) is always an underestimation of the real distance (and makes this algorithm therefore admissible).

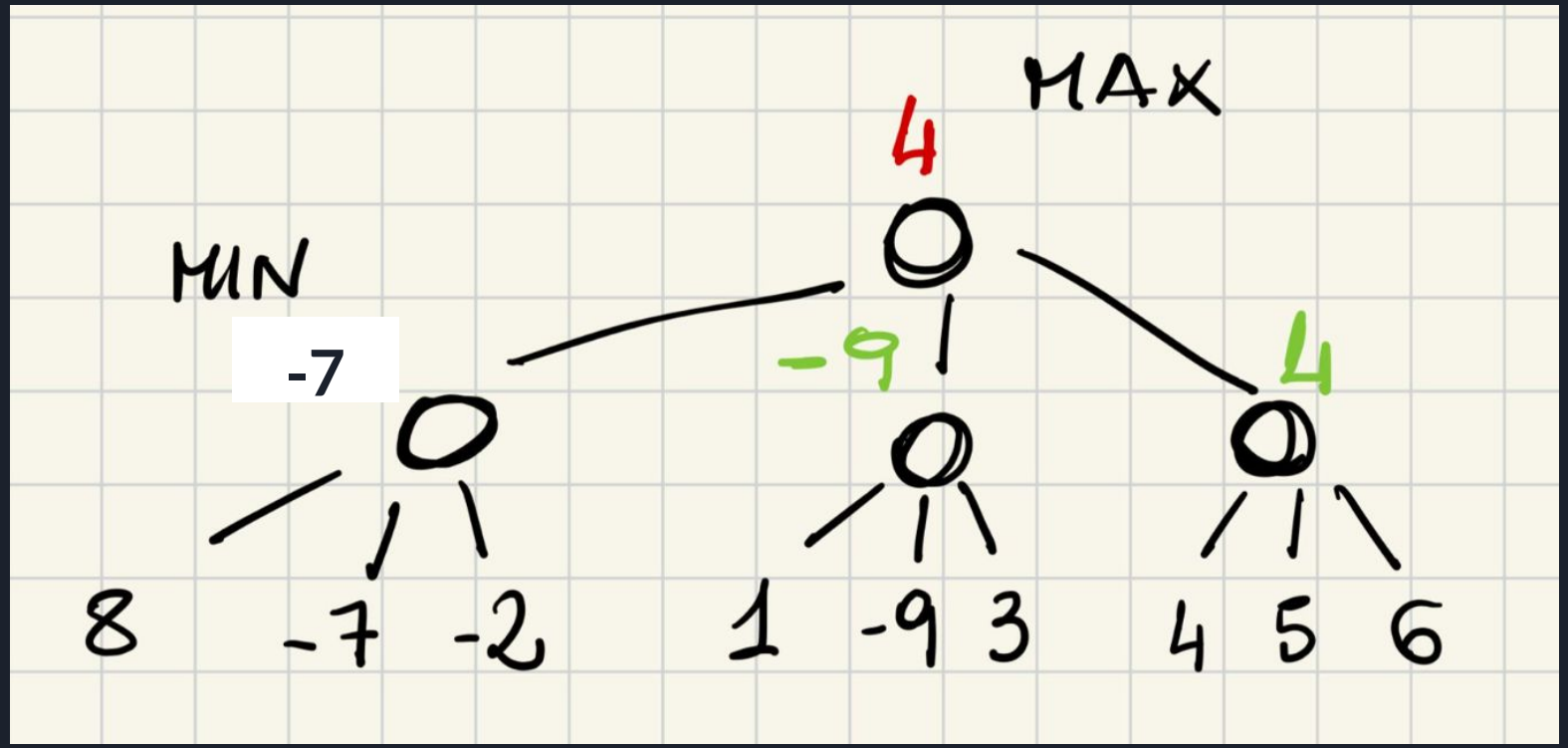
Exercise 7:

Given is the following search tree:

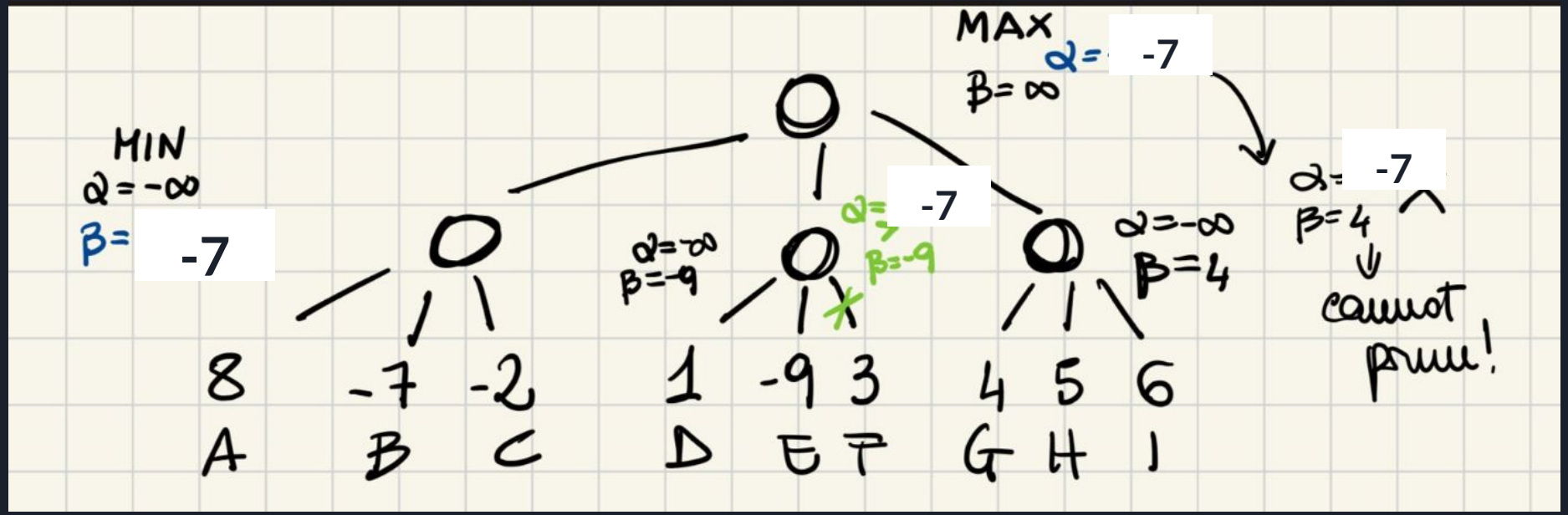


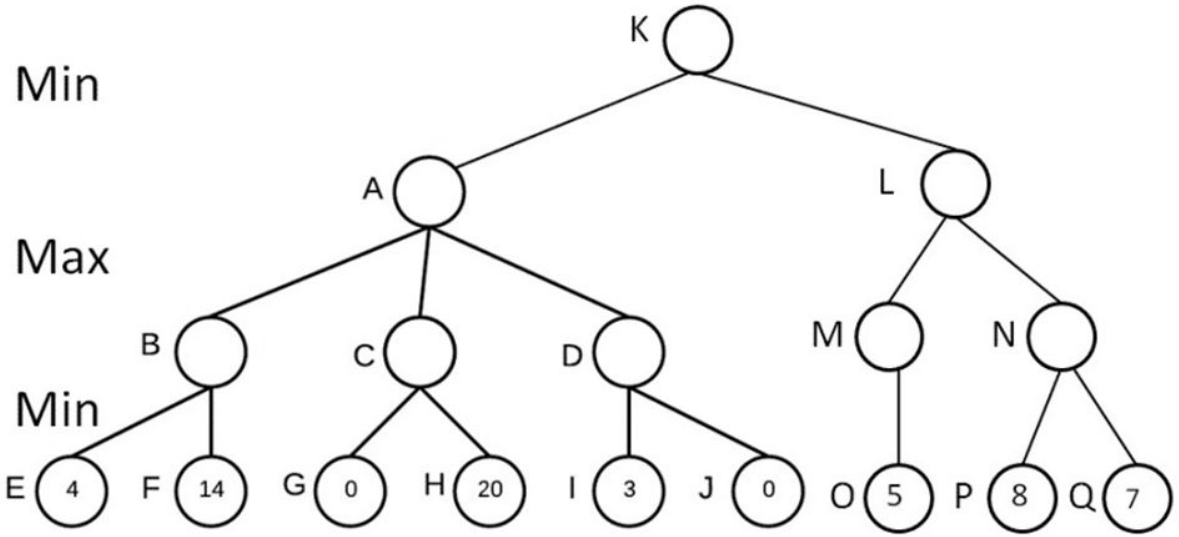
What is the expected value on the root of the tree if Max has the first move? Which nodes can you prune if you use Alpha/Beta pruning?

VALUE ON THE ROOT (MAX PLAYER)

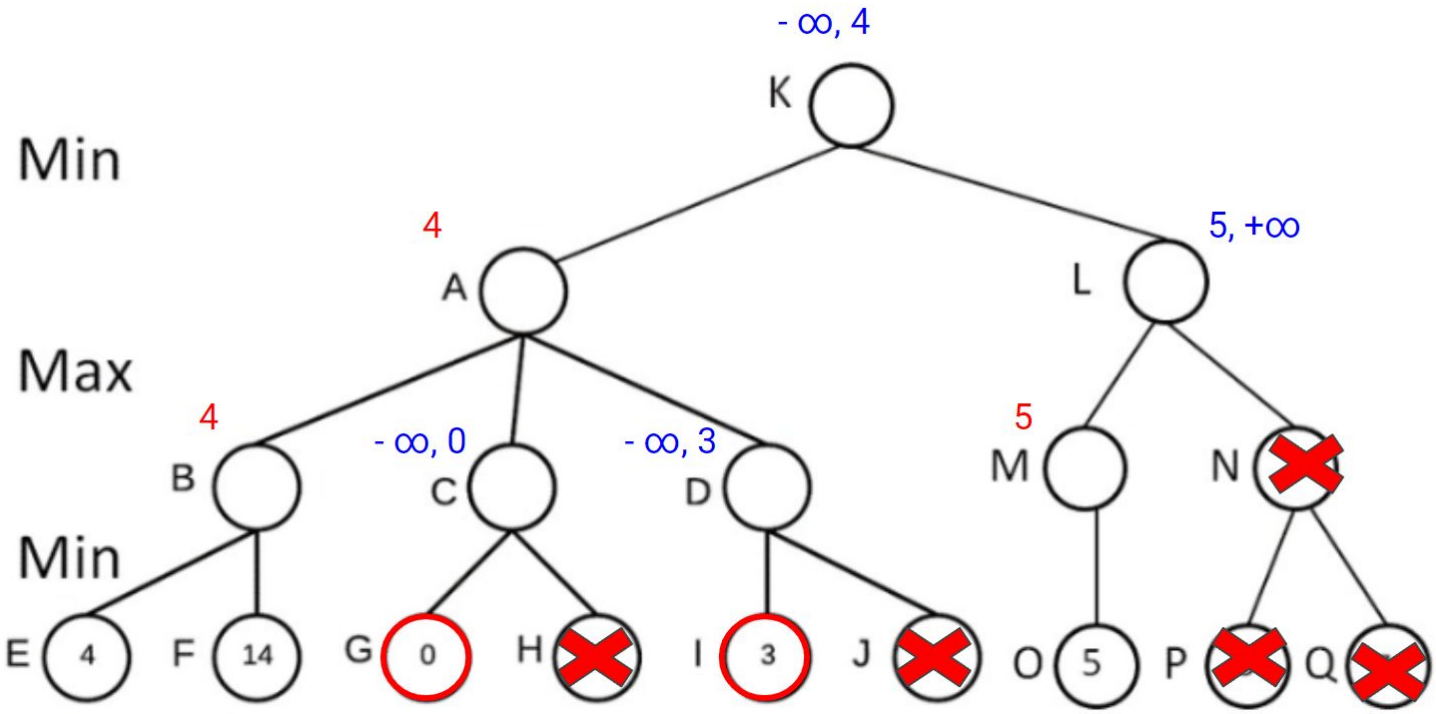


WHICH NODES CAN BE PRUNED?





a. H,J
b. N,P,Q
c. D,I,J
d. H,I
e. M,O
f. L,M,N,O,P,Q


- ∞, 4
K
Min
4
A
5, +∞
L
Max
4
B
- ∞, 0
C
- ∞, 3
D
5
M
N
Min
E
4
F
14
G
0
H
I
3
J
O
5
P
Q

Exercise 12: DPLL for SAT solving

Given a clause set

S = { A v B,

-A v -B,

A v -B v C,

-A v B,

A v -C}

Prove satisfiability, or unsatisfiability, with DPLL. Apply the MOMs heuristics, and assign the value 0 if the “winning literal” occurs with more negation signs than without, and 1 otherwise (equal or less). In case of a draw use alphabetical order, and assign 0 first.

$$S = \{ A \vee B, -A \vee -B, A \vee -B \vee C, -A \vee B, A \vee -C \}$$

- A is the literal that occurs the most, and most of the times it has a positive value.
- A = 1
- S = {-B, B}
- CONTRADICTION: S is unsatisfiable

Exercise 13: DPLL for Entailment

Prove, or disprove whether **C -> -(A v -B)** and **A** entail **-C** or not, using DPLL with alphabetical clause selection, and negation first.

We define a procedure **KB ⊢ a** {

1. Translate KB into CNF to get **cnf(KB)**
2. Translate ¬a into CNF to get **cnf(¬a)**
3. Add cnf(¬a) to cnf(KB)
4. Apply SAT solver (such as DPLL) on **cnf(KB) ∪ cnf(¬ a)** until
   - either model is found (satisfiable) or
   - search exhausted (unsatisfiable)
5. If **satisfiable**, return "**NOT ENTAILED**", otherwise "**ENTAILED**".

STEP 1: TURN KB CLAUSES INTO CNF

- C v - (A v -B)
- C v (-A & B)
(-C v -A) & (-C v B)

STEP 2: NEGATE THE FORMULA -C

-C IS TURNED INTO -(-C), THAT IS **C**

- **(-C v -A)**
- **(-C v B)**
- **A**
- **C**

- (-C v -A)
- (-C v B)
- A
- C

1) A is a Unit Clause, so A=True

{-C,
-C v B,
C}

2) C is a Unit Clause → CONTRADICTION!

OUR STATEMENT IS UNSATISFIABLE, SO -C IS ENTAILED.